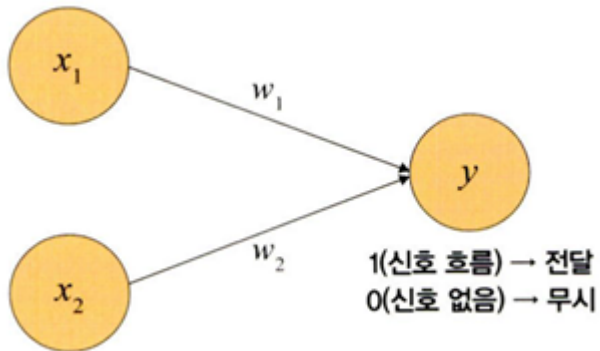


<https://jade0728.tistory.com/61> 에서 확인 가능합니다.

4.1 인공 신경망의 한계와 딥러닝 출현

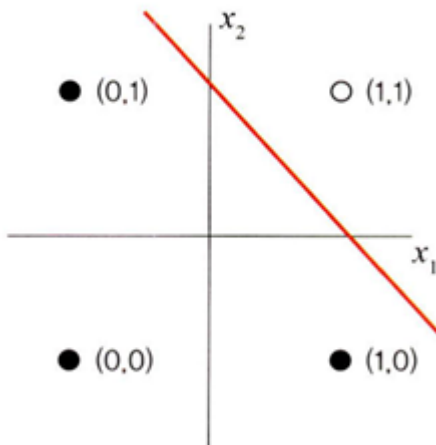
퍼셉트론: 딥러닝의 기원이 되는 알고리즘



퍼셉트론을 논리게이트로 확인해보면 다음과 같다.

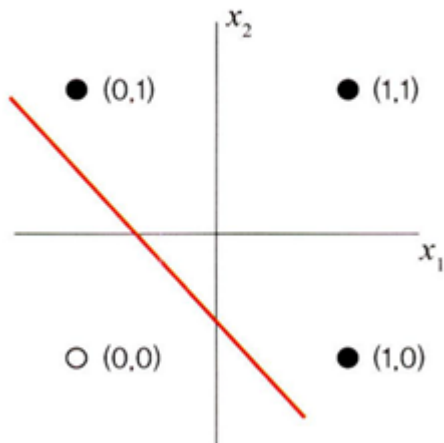
- **AND Gate**: 모든 입력이 1일때만 결과 1을 갖는다.

▼ 그림 4-2 AND 게이트



- **OR Gate**: 둘 중 하나만 1이거나 둘다 1일 때 작동함

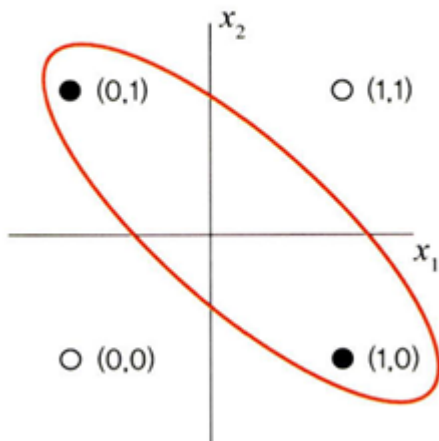
▼ 그림 4-3 OR 게이트



- **XOR Gate**: 입력 두개 중 한개만 1일 때 작동, 데이터가 비선형적으로 분리되기 때문에 제대로 된 분류가 어렵다.

이를 극복하는 방안으로 **hidden layer**를 두어 비선형적으로 분리되는 데이터에 대해서도 학습이 가능하도록 다층 퍼셉트론을 고안하게 되었다. 여러 **hidden layer**가 있는 신경망을 심층 신경망(**DNN=Deep Neural Network**)이라고 하며 딥러닝이라고도 부른다.

▼ 그림 4-4 XOR 게이트

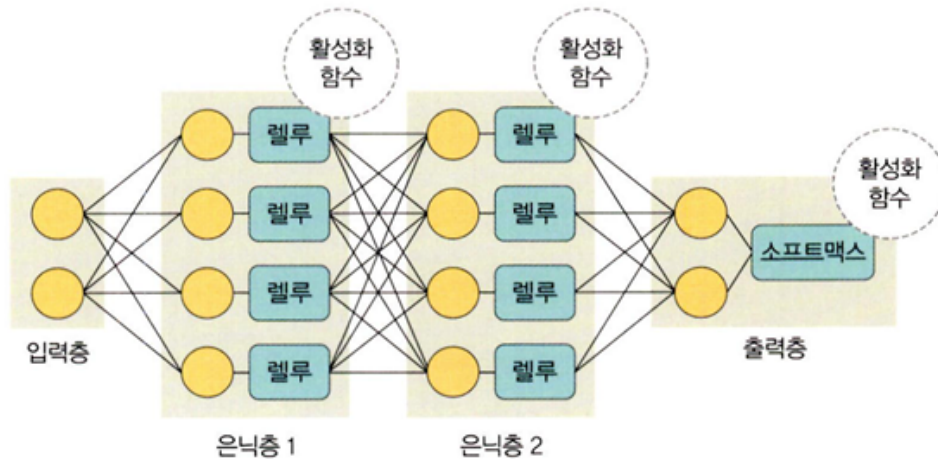


4.2 딥러닝 구조

4.2.1 딥러닝 용어

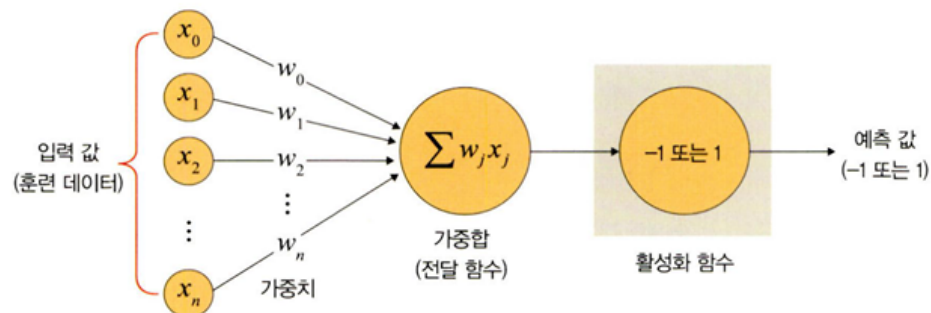
딥러닝은 1) 입력층 **input layer** 2) 은닉층 **hidden layer** 3) 출력층 **output layer**로 구성된다.

♥ 그림 4-5 딥러닝 구조



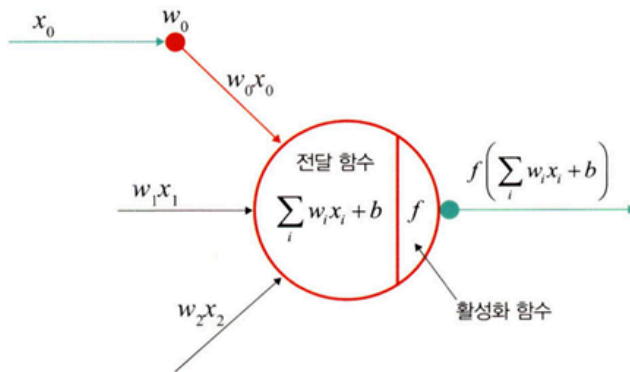
가중치 **weight**: 가중치는 입력값이 연산 결과에 미치는 영향력을 조절하는 요소이다.

♥ 그림 4-6 가중치



가중합, 전달함수: 각 노드에서 들어오는 신호에 **weight**를 곱해서 다음 노드로 전달한다. 이 값들을 모두 더한 합을 가중합이라고 한다. 이 값들을 활성화 함수(activation)으로 보내기 때문에 전달함수(transfer function)이라고도 한다.

♥ 그림 4-7 전달 함수



가중합을 구하는 공식은 다음과 같습니다.

$$\sum_i w_i x_i + b$$

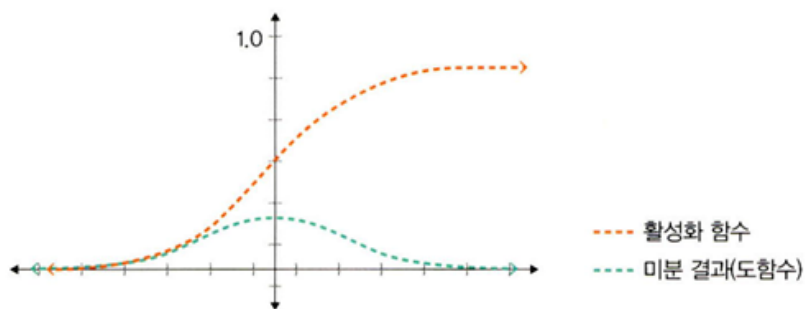
(w : 가중치, b : 바이어스)

활성화 함수 **activation function**: 활성화 함수는 전달 함수에서 전달받은 값을 일정 기준에 따라 출력값을 변화시키는 비선형 함수이다. **sigmoid** 함수, **hyperbolic tangent** 함수, **ReLU** 함수 등이 있다.

1) sigmoid 함수: 함수의 결과를 0~1 사이의 값으로 변형한다. 로지스틱 회귀와 같은 분류문제를 확률적으로 표현하는데 사용한다. 딥러닝 모델의 깊이가 깊어지면 **vanishing gradient**가 발생하여 딥러닝에서는 잘 사용하지 않는다.

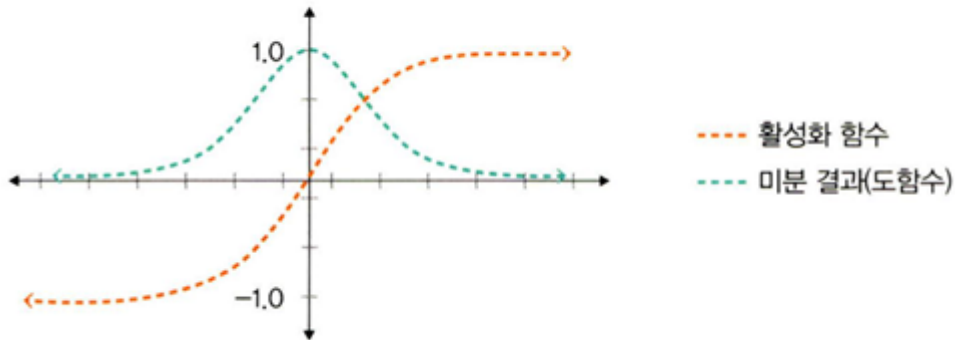
$$f(x) = \frac{1}{1 + e^{-x}}$$

♥ 그림 4-8 시그모이드 활성화 함수와 미분 결과



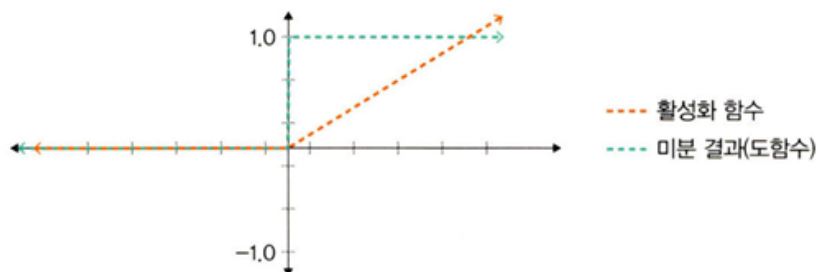
2) hyperbolic tangent 함수: 함수의 결과를 $-1 \sim 1$ 사이의 비선형 형태로 변형한다. 여기에서도 vanishing gradient 문제는 여전히 발생한다.

▼ 그림 4-9 하이퍼볼릭 탄젠트 활성화 함수와 미분 결과



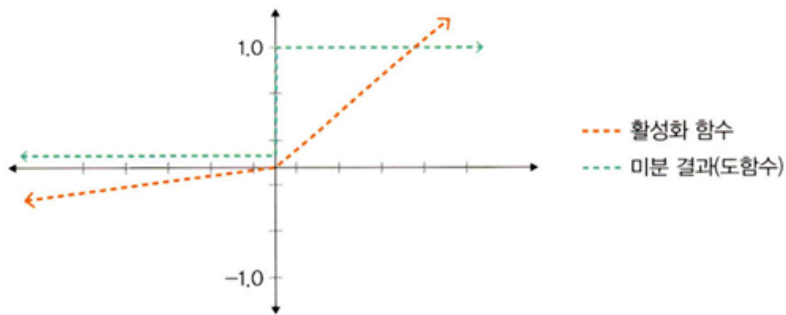
3) ReLU 함수: 입력(x)이 음수면 0을 출력하고, 양수일 때는 x 를 출력한다. gradient descent에 영향을 주지 않아 학습 속도가 빠르고, vanishing gradient가 발생하지 않는다는 장점이 있다. 보통 hidden layer에서 사용된다. 단점은 음수 값을 받으면 항상 0을 출력하기 때문에 학습 능력이 감소될 수 있다.

▼ 그림 4-10 렐루 활성화 함수와 미분 결과



4) Leaky ReLU 함수: ReLU의 단점을 보완한 함수로, 입력값이 음수이면 0이 아닌 매우 작은 수를 반환한다.

♥ 그림 4-11 리키 렐루 활성화 함수와 미분 결과



5) softmax 함수: 입력 값을 0~1 사이에서 출력되도록 정규화하여 출력 값들의 총합이 항상 1이 되도록 한다. 딥러닝 출력 노드의 활성화 함수로 많이 사용된다.

$$y_k = \frac{\exp(a_k)}{\sum_{i=1}^n \exp(a_i)}$$

다음은 손실함수이다.

경사하강법은 학습률(learning rate)과 손실함수의 순간 기울기를 이용하여 가중치를 업데이트하는 방식이다. 미분의 기울기를 이용하여 오차를 최소화하는 방향으로 이동시키는 방법인데, 이때 오차를 구하는 방법이 손실함수이다.

대표적으로 평균 제곱 오차(Mean Squared Error = MSE)와 크로스 엔트로피 오차(Cross Entropy Error, CEE)가 있다.

1) 평균 제곱 오차 MSE

실제 값과 예측 값의 차이를 제곱하여 평균을 낸 것이 평균 제곱 오차(MSE)이다. 보통 회귀에서 손실함수로 주로 사용한다.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\left(\begin{array}{l} \hat{y}_i: \text{신경망의 출력(신경망이 추정한 값)} \\ y_i: \text{정답 레이블} \\ i: \text{데이터의 차원 개수} \end{array} \right)$$

2) 크로스 엔트로피 오차 CEE

분류문제에서 **one-hot encoding**했을 때만 사용할 수 있는 오차 계산법이다.
 일반적으로 분류문제에서 데이터의 출력을 0과 1로 구분하기 위해 **sigmoid** 함수를
 사용하는데, **MSE**를 사용하면 울퉁불퉁한 그래프가 출력된다. **cee**를 사용하면
 경사하강법 과정에서 학습이 **local minimum**에서 멈출 수 있다.

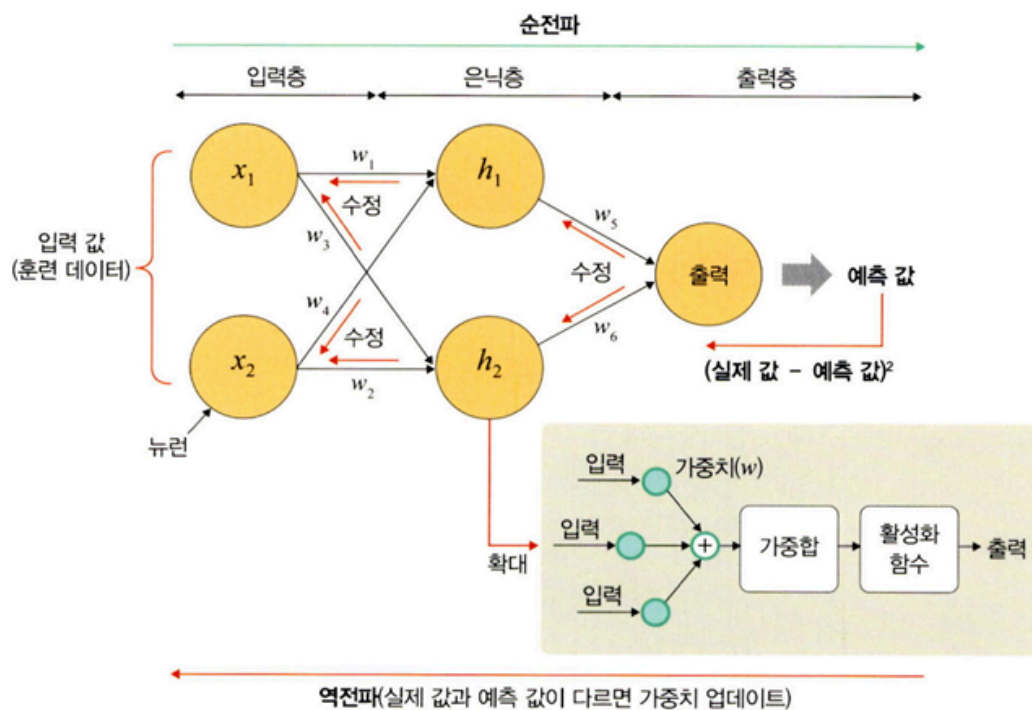
$$CrossEntropy = - \sum_{i=1}^n y_i \log \hat{y}_i$$

$\hat{y}_i$: 신경망의 출력(신경망이 추정한 값)
 y_i : 정답 레이블
 i : 데이터의 차원 개수

4.2.2 딥러닝 학습

딥러닝 학습은 **feedforward**(순전파)와 **backpropagation**(역전파)로 구성된다.

▼ 그림 4-12 순전파와 역전파



1) feedforward(순전파): 이전 층의 뉴런에서 수신한 정보에 가중합/활성화 함수를
 적용하여 다음층의 뉴런으로 전송하는 방식

2) backpropagation(역전파): 손실 함수 비용을 0에 가깝도록 하기 위해 모델이 훈련을 반복하면서 가중치를 조정한다. 출력층 -> 은닉층 -> 입력층으로 전달되기 때문에 역전파라고 한다.

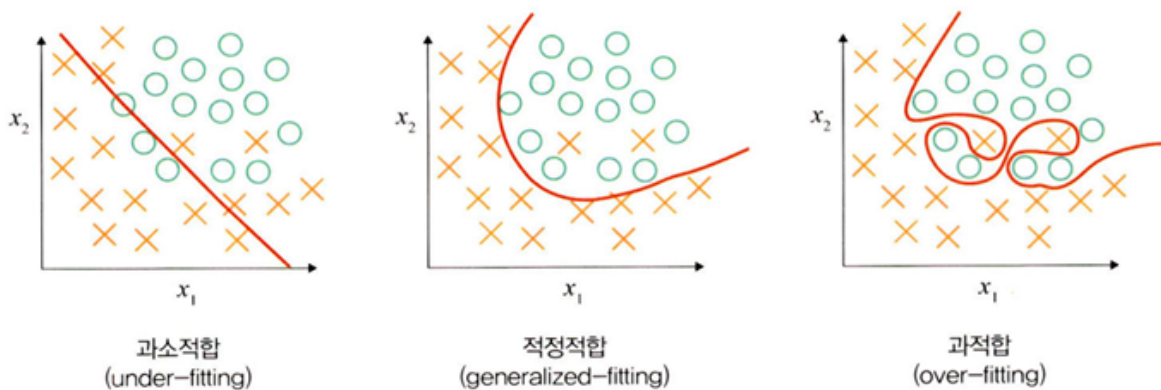
4.2.3 딥러닝의 문제점과 해결방안

딥러닝은 **1) overfitting(과적합)**, **2) vanishing gradient(기울기 소멸 문제)**, **3) 성능이 나빠지는 문제**가 발생할 수 있다.

1) overfitting 과적합

데이터를 과하게 학습해서 발생한다. 훈련 데이터에 과하게 학습하여 실제 검증 데이터에 대한 오차가 증가하는 현상이다.

▼ 그림 4-14 과적합



이 문제를 해결하기 위해 **dropout** 기법을 사용한다. dropout은 학습 과정 중에 임의로 일부 노드들을 학습에서 제외시키는 방법이다.

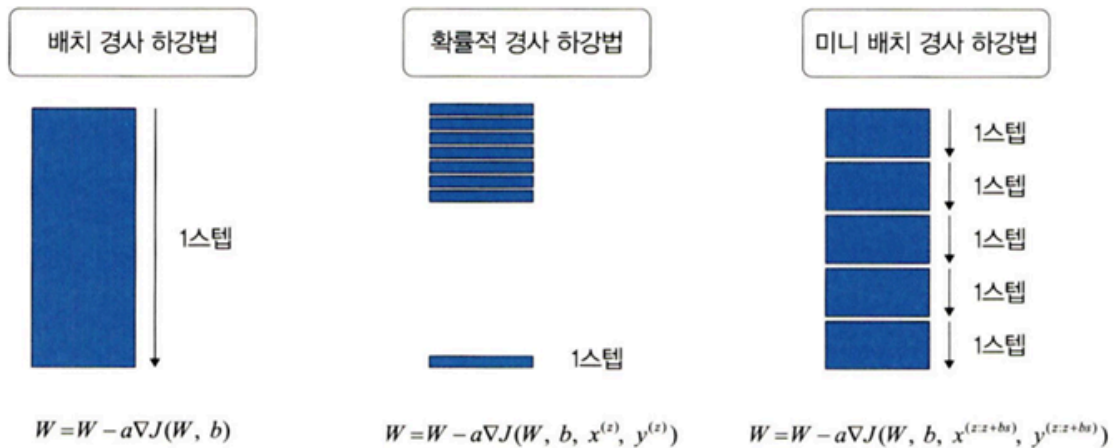
2) vanishing gradient: 기울기 소멸 문제

보통 hidden layer에서 발생하는데, 기울기가 소멸되어 학습되는 양이 0에 가까워져 학습이 거의 진행되지 않는 상태를 의미한다.

3) 성능이 나빠지는 문제

경사 하강법은 손실 함수의 비용이 최소가 되는 지점을 찾을 때까지 기울기가 낮은 쪽으로 계속 이동시키는 과정을 반복하는데, 이때 성능이 나빠지는 문제가 발생한다. 이 문제를 해결하기 위해 확률적 경사 하강법과 미니배치 경사 하강법을 사용한다.

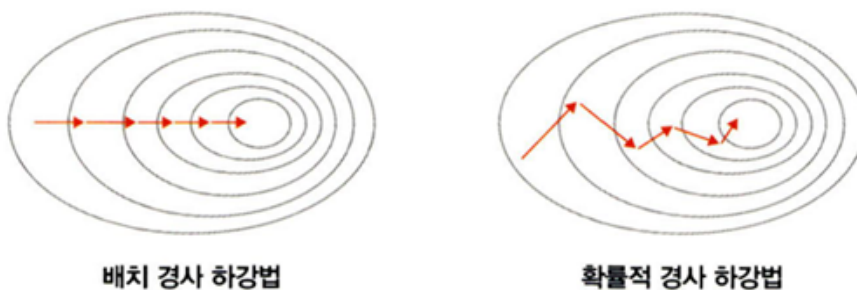
▼ 그림 4-18 경사 하강법의 유형



배치 경사 하강법은 전체 데이터셋에 대한 오류를 구한 이후, 기울기를 한번만 계산하여 모델의 파라미터를 업데이트하는 방법이다. 단점은 한 스텝에 모든 훈련 데이터셋을 사용해서 학습이 오래 걸린다는 점인데, 이를 개선한 것이 확률적 경사 하강법이다.

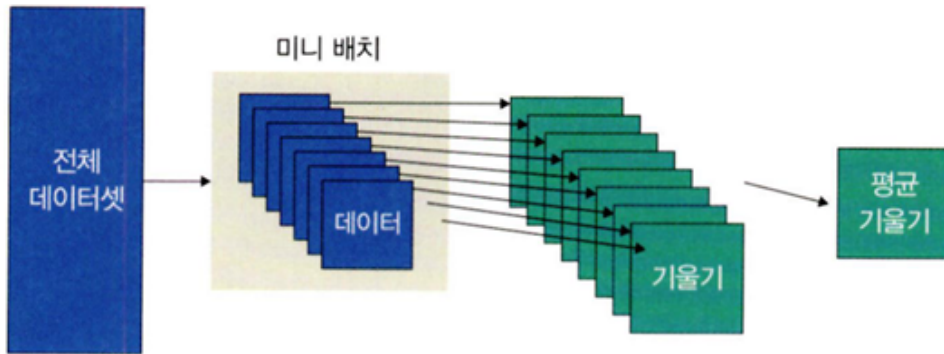
확률적 경사 하강법(**Stochastic Gradient Descent, SGD**)는 임의로 선택한 데이터에 대해 기울기를 계산하는 방법으로 적은 데이터를 사용하기 때문에 빠른 계산이 가능하다.

▼ 그림 4-19 배치 경사 하강법과 확률적 경사 하강법



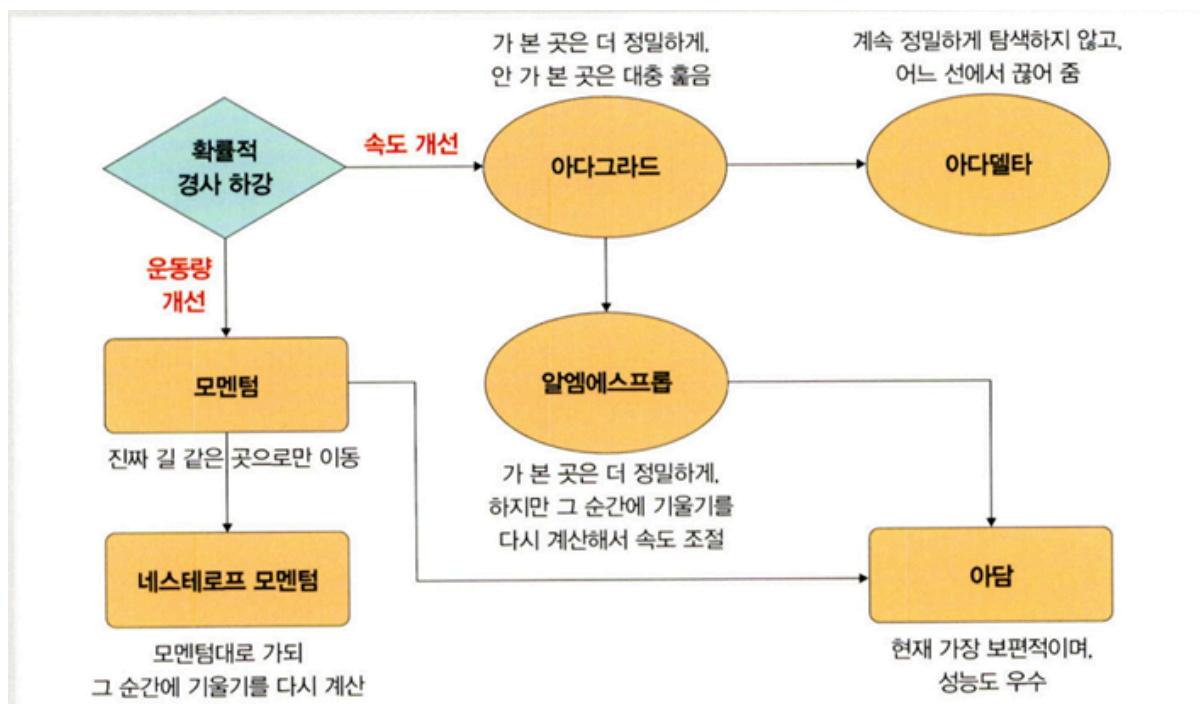
미니 배치 경사 하강법은 전체 데이터셋을 미니 배치 여러 개로 나누고, 미니 배치 한개마다 기울기를 구한 후 그것의 평균 기울기를 이용하여 모델을 업데이트하는 방식이다.

▼ 그림 4-20 미니 배치 경사 하강법



+ 옵티마이저 optimizer

확률적 경사 하강법의 파라미터 변경 폭이 불안정한 문제를 해결하기 위해 학습 속도와 운동량을 조절하는 **optimizer**를 도입할 수 있다.



1) 속도를 조절하는 방법

1.1) 아다그라드 Adagrad, Adaptive Gradient

변수의 업데이트 횟수에 따라 학습률을 조절한다. 많이 변화하지 않는 변수들의 학습률은 크게 하고, 많이 변화하는 변수들의 학습률은 작게 한다.

파라미터마다 다른 학습률을 주기 위해 **G**함수를 도입한다. **G**값은 이전 **G**값의 누적이다.

1.2) 아다델타 **Adadelta, Adaptive Delta**

아다그라드에서 **G**값이 커짐에 따라 학습이 멈추는 문제를 해결하기 위해 등장한 방법이다. 아다그라드 수식에서 학습률을 **D**함수로 변환했기 때문에 학습률에 대한 하이퍼파라미터가 필요하지 않다.

1.3) RMSProp

아다그라드의 **G**값이 무한히 커지는 것을 방지하고자 제안된 방법이다.

아다그라드에서 학습이 안되는 문제를 해결하기 위해 **G**함수에서 **r**이 추가된다.

G값이 너무 크면 학습이 안될 수 있어서 **r** 값을 통해 학습의 크기를 조절할 수 있다.

2) 운동량을 조절하는 방법

2.1) 모멘텀 **Momentum**

가중치를 수정하기 전에 이전 수정방향을 참고하여 같은 방향으로 일정한 비율만 수정하는 방법이다. 지그재그 현상을 줄이고 관성효과를 얻을 수 있다.

2.2) 네스테로프 모멘텀 **Nesterov Accelerated Gradient, NAG**

모멘텀은 멈춰야할 시점에서도 관성에 의해 더 멀리 갈 수 있는 단점이 있지만, 네스테로프 방법은 모멘텀으로 절반 정도 이동한 이후 어떤 방식으로 이동해야 하는지 다시 계산하여 결정한다.

3) 속도와 운동량에 대한 혼용 방법

3.1) 아담 **Adam, Adaptive Moment Estimation**

모멘텀과 알엠에스프롭의 장점을 결합한 방법이다.

4.2.4 딥러닝의 이점

1) 특성 추출 **feature extraction**

hidden layer를 깊게 쌓는 방식으로 파라미터를 늘렸고, 이를 통해 데이터의 특성을 잘 잡아낼 수 있게 되었다.

2) 빅데이터의 효율적 활용

데이터 사례가 많을수록 특성추출이 잘 된다. 반대로, 확보된 데이터가 적다면 딥러닝의 성능 향상을 기대하기 힘들기 때문에 머신러닝을 고려해 봐야 한다.

4.3 딥러닝 알고리즘

딥러닝 알고리즘은 목적에 따라 CNN, RNN, RBM, DBN 으로 분류된다.

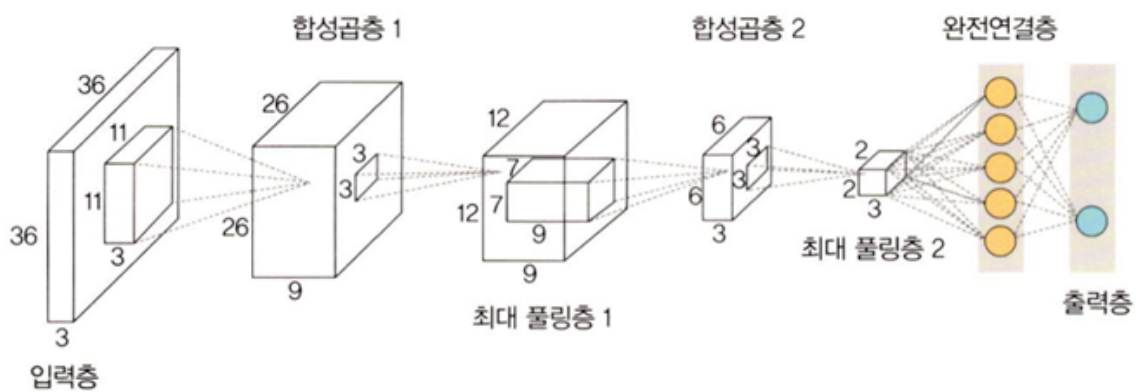
4.3.1 심층 신경망 DNN

입력층과 출력층 사이에 다수의 Hidden layer를 포함하는 인공 신경망

4.3.2 CNN, 합성곱 신경망

Convolution layer와 pooling layer를 포함하며 이미지 처리 성능이 좋은 인공 신경망 알고리즘이다. 이미지 데이터에서 객체를 탐색하거나 객체 위치를 찾아내는 데 유용한 신경망이다.

▼ 그림 4-25 합성곱 신경망

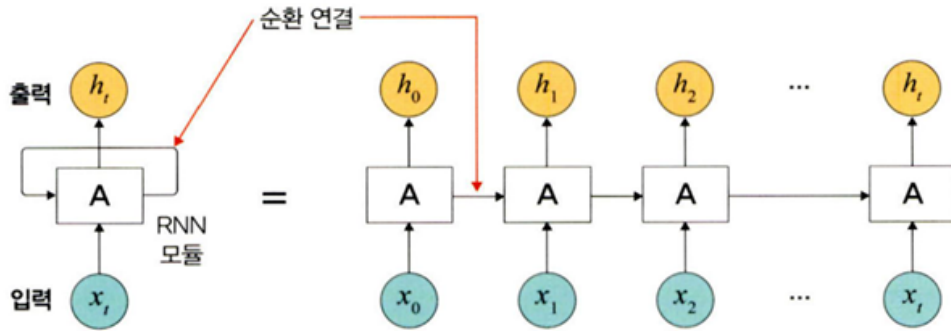


CNN의 종류로는 LeNet-5, AlexNet, VGG, GoogLeNet, ResNet 등이 존재한다.

4.3.3 RNN, 순환 신경망

시계열 데이터(음악, 영상 등) 같은 시간 흐름에 따라 변화하는 데이터를 학습하기 위한 인공 신경망이다.

▼ 그림 4-26 순환 신경망



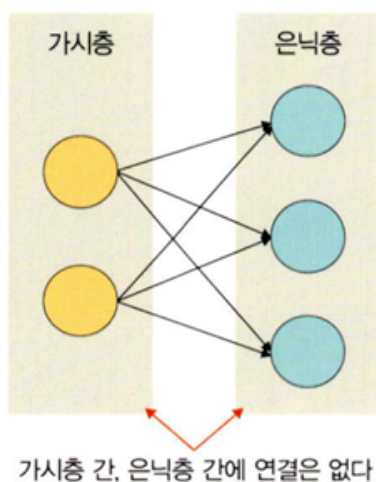
Vanishing gradient problem으로 인해 학습이 되지 않는 문제가 발생하는데, 이를 해결하고자 LSTM이 많이 사용되고 있다.

4.3.4 RBM, 제한된 볼츠만 머신

RBM은 Visible layer(가시층)과 Hidden Layer(은닉층)으로 구성된 모델이다. 이 모델에서 가시층은 은닉층과만 연결된다.

차원 감소, 분류, 선형 회귀 분석, 협업 필터링(collaborative filtering), 특성 값 학습(feature learning), 주제 모델링(topic modeling)에 사용된다. DBN의 요소로 활용된다.

▼ 그림 4-27 제한된 볼츠만 머신

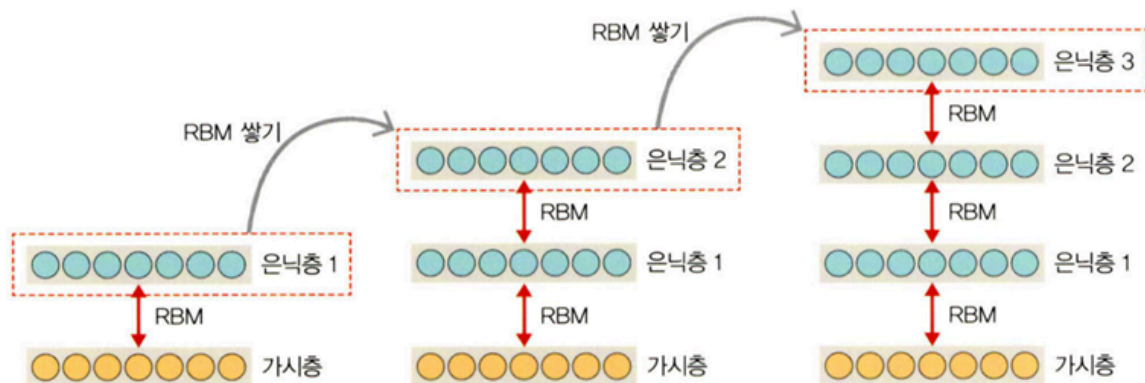


4.3.5 DBN 심층 신뢰 신경망

RBM을 여러 층으로 쌓은 형태이다. 사전 훈련된 RBM을 층층이 쌓아올린 구조로, 레이블이 없는 데이터에 대한 비지도 학습이 가능하다.

비지도 학습으로 학습하며, 위로 올라갈수록 추상적 특성을 추출한다. 순차적으로 심층 신뢰 신경망을 학습시켜 계층적 구조를 생성한다.

▼ 그림 4-28 심층 신뢰 신경망



4.4 마무리(우리는 무엇을 배워야 할까)

간단한 분류 정도는 머신러닝만으로 충분하지만, 복잡한 비선형 데이터에 대한 분류 및 예측을 하고 싶다면 딥러닝으로 학습해야 한다. 앞으로는 딥러닝 도구를 더 자세히 알아보도록 한다.